

A Practical Guide to Long short-term memory

TOPIC -- LONG SHORT-TERM MEMORY

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$h_t = o_t \odot \tanh(c_t)$$

-- 01 --

Long short-term memory (LSTM) is a type of recurrent neural network (RNN) aimed at mitigating the vanishing gradient problem commonly encountered by traditional RNNs. Its relative insensitivity to gap length is its advantage over other RNNs, hidden Markov models, and other sequence learning methods. It aims to provide a short-term memory for RNN that can last thousands of timesteps (thus "long short-term memory"). The name is made in analogy with long-term memory and short-term memory and their relationship, studied by cognitive psychologists since the early 20th century. An LSTM unit is typically composed of a cell and three gates: an input gate, an output gate, and a forget gate. The cell remembers values over arbitrary time intervals, and the gates regulate the flow of information into and out of the cell. Forget gates decide what information to discard from the previous state, by mapping the previous state and the current input to a value between 0 and 1. A (rounded) value of 1 signifies retention of the information, and a value of 0 represents discarding. Input gates decide which pieces of new information to store in the current cell state, using the same system as forget gates. Output gates control which pieces of information in the current cell state to output, by assigning a value from 0 to 1 to the information, considering the previous and current states. Selectively outputting relevant information from the current state allows the LSTM network to maintain useful, long-term dependencies to make predictions, both in current and future time-steps. LSTM has wide applications in classification, data processing, time series analysis tasks, speech recognition, machine translation, speech activity detection, robot control, video games, healthcare, energy forecasting.

-- 02 --

$$\begin{aligned}
 f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
 i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
 o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
 c_{\sim t} &= \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
 c_t &= f_t \blacksquare c_{\sim t} - 1 + i_t \blacksquare c_{\sim t} h_t = o_t \blacksquare \sigma_h(c_t)
 \end{aligned}$$

$$\begin{aligned}
 f_t &= \sigma_g(W_f x_t + U_f h_{t-1} + b_f) \\
 i_t &= \sigma_g(W_i x_t + U_i h_{t-1} + b_i) \\
 o_t &= \sigma_g(W_o x_t + U_o h_{t-1} + b_o) \\
 c_{\sim t} &= \sigma_c(W_c x_t + U_c h_{t-1} + b_c) \\
 c_t &= f_t \odot c_{\sim t} - 1 + i_t \odot c_{\sim t} h_t = o_t \odot \sigma_h(c_t)
 \end{aligned}$$

-- 03 --

CTC score function Many applications use stacks of LSTM RNNs and train them by connectionist temporal classification (CTC) to find an RNN weight matrix that maximizes the probability of the label sequences in a training set, given the corresponding input sequences. CTC achieves both alignment and recognition.

-- 04 --

Further reading Monner, Derek D.; Reggia, James A. (2010). "A generalized LSTM-like training algorithm for second-order recurrent neural networks" (PDF). *Neural Networks*. 25 (1): 70–83. doi:10.1016/j.neunet.2011.07.003. PMC 3217173. PMID 21803542. High-performing extension of LSTM that has been simplified to a single node type and can train arbitrary architectures Gers, Felix A.; Schraudolph, Nicol N.; Schmidhuber, Jürgen (Aug 2002). "Learning precise timing with LSTM recurrent networks" (PDF). *Journal of Machine Learning Research*. 3: 115–143. Gers, Felix (2001). "Long Short-Term Memory in Recurrent Neural Networks" (PDF). PhD thesis. Abidogun, Olusola Adeniyi (2005). *Data Mining, Fraud Detection and Mobile Telecommunications: Call Pattern Analysis with Unsupervised Neural Networks*. Master's Thesis (Thesis). University of the Western Cape. hdl:11394/249. Archived (PDF) from the original on May 22, 2012. original with two chapters devoted to explaining recurrent neural networks, especially LSTM.

-- 05 --

Training An RNN using LSTM units can be trained in a supervised fashion on a set of training sequences, using an optimization algorithm like gradient descent combined with backpropagation through time to compute the gradients needed during the optimization process, in order to change each weight of the LSTM network in proportion to the derivative of the error (at the output layer of the LSTM network) with respect to corresponding weight. A problem with using gradient descent for standard RNNs is that error gradients vanish exponentially quickly with the size of the time lag between important events. This is due to lim

$\lim_{n \rightarrow \infty} W^n = 0$ if the spectral radius of W is smaller than 1. However, with LSTM units, when error values are back-propagated from the output layer, the error remains in the LSTM unit's cell. This "error carousel" continuously feeds error back to each of the LSTM unit's gates, until they learn to cut off the value.

-- 06 --

where the initial values are $c_0 = 0$ and $h_0 = 0$ and the operator $\odot$ denotes the Hadamard product (element-wise product). The subscript t indexes the time step.

-- 07 --

2015: Google started using an LSTM trained by CTC for speech recognition on Google Voice. According to the official blog post, the new model cut transcription errors by 49%. 2016: Google started using an LSTM to suggest messages in the Allo conversation app. In the same year, Google released the Google Neural Machine Translation system for Google Translate which used LSTMs to reduce translation errors by 60%. Apple announced in its Worldwide Developers Conference that it would start using the LSTM for quicktype in the iPhone and for Siri. Amazon released Polly, which generates the voices behind Alexa, using a bidirectional LSTM for the text-to-speech technology. 2017: Facebook performed some 4.5 billion automatic translations every day using long short-term memory networks. Microsoft reported reaching 94.9% recognition accuracy on the Switchboard corpus, incorporating a vocabulary of 165,000 words. The approach used "dialog session-based long-short-term memory". 2018: OpenAI used LSTM trained by policy gradients to beat humans in the complex video game of Dota 2, and to control a human-like robot hand that manipulates physical objects with unprecedented dexterity. 2019: DeepMind used LSTM trained by policy gradients to excel at the complex video game of Starcraft II.

-- 08 --

The figure on the right is a graphical representation of an LSTM unit with peephole connections (i.e. a peephole LSTM). Peephole connections allow the gates to access the constant error carousel (CEC), whose activation is the cell state. h_{t-1} is not used, c_{t-1} is used instead in most places.